

Section 1 - Market-Level Analysis

Rocky Rowell

2026-06-09

Description

This file is one of the analysis files in the project. All market-level analysis is conducted in this file. The first form of analysis is a set of exploratory graphs observing each outcome variable over the sample period, grouped by game-age. The second form of analysis is a set of graphs comparing outcome variables for Steam-only, Dual-Platform, and Future Dual-Platform games after Epic's launch, also grouped by game-age. The final form of analysis is a set of regression models that examine the change in outcome variable trajectories for Steam-only and Dual-Platform games relative to 2018. These regression models are visualized through a set of graphs.

Notes / Important Information

- Although the No-Game-FE models are created in this file, they are not used in the thesis.
 - 2013 dropped from graphs and regressions (low sample size = high volatility). 2013 only includes a few weeks of data in this sample, which messes with the graphs.
 - Regression models account for time trends and seasonality with $\log(\text{Quarters Since Release})$ and month-of-year fixed effects
-

Setup and Data Cleaning

```
# Libraries
library(tidyverse)
library(fixest)
library(ggthemes)
library(patchwork)
library(modelsummary)
library(flextable)

# Data
data <- read_csv("Data/cleaned_data.csv")
```

Shared Globals

```
epic_launch <- as.Date("2018-12-01")

# updated game age windows: 0-6mo, 6mo-2yr, 2yr+
graph_windows <- list(
```

```

"0-6 Months" = c(0, 182),
"6 Months - 2 Years" = c(182, 730),
"2+ Years" = c(730, Inf)
)

genre_cols <- c("RPG", "Action", "Adventure", "Shooter", "Strategy", "Simulation",
               "Platformer", "Puzzle", "Horror", "OpenWorld", "Survival",
               "Roguelike", "Multiplayer", "Narrative")

```

Epic Entry Dates

```

epic_entry_dates <- data %>%
  filter(shop_name == "Epic") %>%
  group_by(game_title) %>%
  summarise(entry_date = min(date), .groups = "drop")

```

Section 1.1: Full Sample Exploratory Graphs

Monthly average Steam prices, sale frequency, and discount depth. Three game-age windows (0-6 months, 6 months to 2 years, 2+ years). Includes 95% CI ribbons around the trend lines.

Helper: build monthly panel for a given age range

```

build_window_panel <- function(age_range, min_frac = 0.05) {
  panel <- data %>%
    filter(shop_name == "Steam") %>%
    mutate(days_since_release = as.numeric(date - release_date)) %>%
    filter(days_since_release >= age_range[1],
           days_since_release < age_range[2]) %>%
    mutate(month = floor_date(date, "month")) %>%
    filter(month < as.Date("2024-12-01")) %>%
    group_by(month) %>%
    summarise(
      avg_real_price = mean(real_price, na.rm = TRUE),
      avg_sale_freq = mean(sale, na.rm = TRUE),
      avg_disc_depth = mean(discount_percent, na.rm = TRUE),
      n_games = n_distinct(game_title),
      .groups = "drop"
    ) %>%
    filter(year(month) > 2013)

  thresh <- median(panel$n_games) * min_frac
  panel %>%
    filter(n_games >= thresh) %>%
    mutate(period = if_else(month < epic_launch, "Pre-Epic", "Post-Epic"))
}

```

Helper: pre/post period averages for annotation

```
compute_period_avgs <- function(df) {  
  df %>%  
    group_by(period) %>%  
    summarise(  
      avg_real_price = mean(avg_real_price, na.rm = TRUE),  
      avg_sale_freq  = mean(avg_sale_freq,  na.rm = TRUE),  
      avg_disc_depth = mean(avg_disc_depth, na.rm = TRUE),  
      label_x        = mean(month),  
      .groups        = "drop"  
    )  
}
```

Helper: single-outcome plot for one window (with 95% CI)

```
plot_window_outcome <- function(df, period_avgs, outcome_col, y_lab,  
                                window_label, label_format = "raw") {  
  label_df <- period_avgs %>%  
    mutate(  
      lbl = case_when(  
        label_format == "dollar" ~ paste0("Avg: $", round(.data[[outcome_col]], 2)),  
        label_format == "percent" ~ paste0("Avg: ", round(.data[[outcome_col]], 2), "%"),  
        TRUE ~ paste0("Avg: ", round(.data[[outcome_col]], 2))  
      )  
    )  
  
  ggplot(df, aes(x = month, y = .data[[outcome_col]])) +  
    geom_line(color = "steelblue", linewidth = 0.6) +  
    geom_smooth(aes(group = period), method = "lm", se = TRUE,  
                color = "firebrick", fill = "firebrick",  
                alpha = 0.15, linewidth = 0.9) +  
    geom_vline(xintercept = epic_launch,  
               linetype = "dashed", color = "black") +  
    geom_label(data = label_df,  
              aes(x = label_x, y = .data[[outcome_col]], label = lbl),  
              color = "firebrick", size = 3, fill = "white") +  
  
    labs(  
      subtitle = window_label,  
      x = NULL, y = y_lab  
    ) +  
    theme_fivethirtyeight() +  
    theme(  
      plot.subtitle = element_text(face = "bold", hjust = 0.5),  
      plot.background = element_rect(fill = "white", color = NA),  
      panel.background = element_rect(fill = "#F7F7F7", color = NA)  
    )  
}
```

Build all three window datasets and period averages

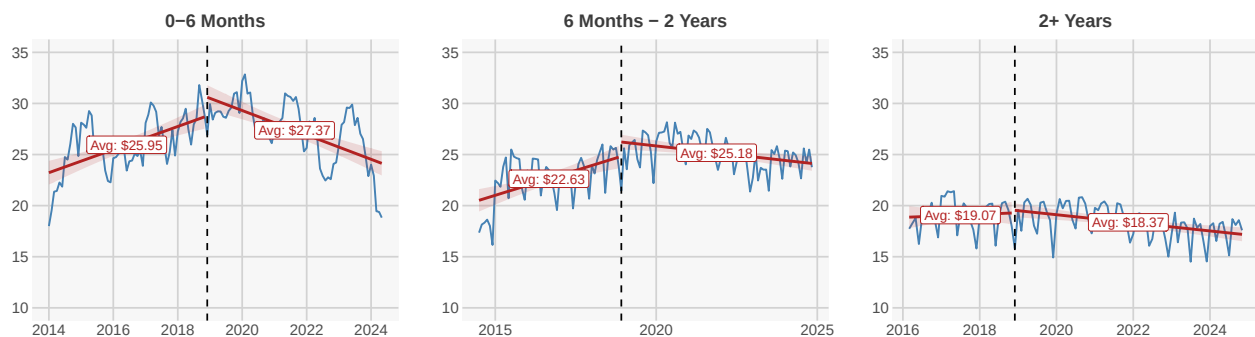
```
window_data <- map(graph_windows, build_window_panel)  
window_avgs <- map(window_data, compute_period_avgs)
```

Real Price Plots

```
p_price <- map2(names(window_data), window_data, function(nm, df) {
  plot_window_outcome(df, window_avgs[[nm]], "avg_real_price",
    "Avg Real Price (USD)", nm, label_format = "dollar")
})

(p_price[[1]] | p_price[[2]] | p_price[[3]]) +
  plot_annotation(
    title = "Monthly Average Real Price on Steam",
    subtitle = "By game age window | dashed = Epic launch"
  ) &
  coord_cartesian(ylim = c(10, 35))
```

Monthly Average Real Price on Steam
By game age window | dashed = Epic launch

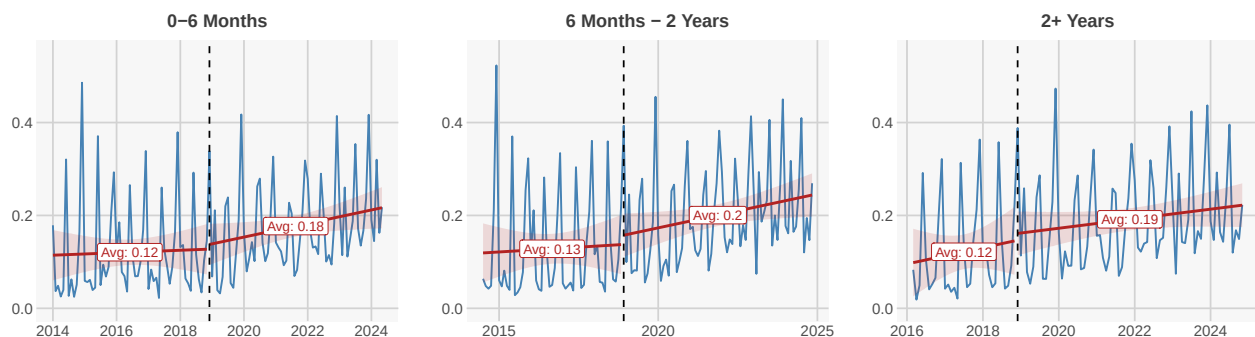


Sale Frequency Plots

```
p_sale <- map2(names(window_data), window_data, function(nm, df) {
  plot_window_outcome(df, window_avgs[[nm]], "avg_sale_freq",
    "Avg Sale Frequency", nm, label_format = "raw")
})

(p_sale[[1]] | p_sale[[2]] | p_sale[[3]]) +
  plot_annotation(
    title = "Monthly Average Sale Frequency on Steam",
    subtitle = "By game age window | dashed = Epic launch"
  ) &
  coord_cartesian(ylim = c(0, .55))
```

Monthly Average Sale Frequency on Steam
By game age window | dashed = Epic launch

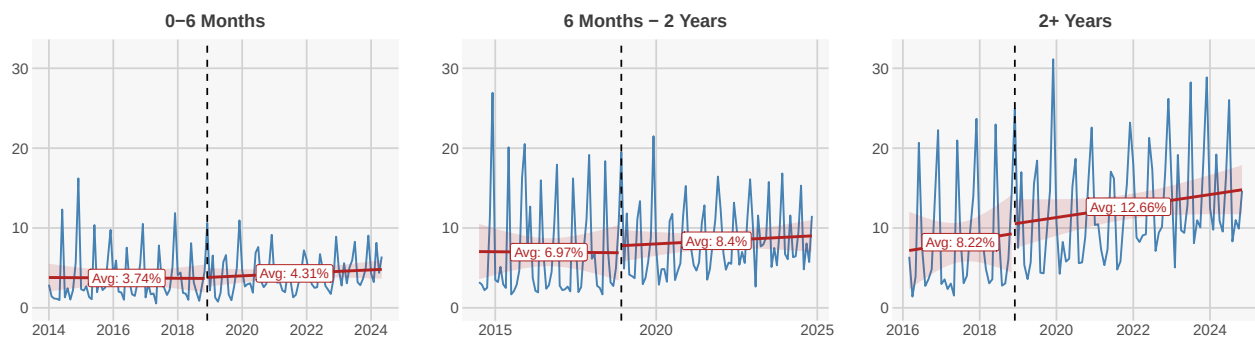


Discount Depth Plots

```
p_disc <- map2(names(window_data), window_data, function(nm, df) {
  plot_window_outcome(df, window_avgs[[nm]], "avg_disc_depth",
    "Avg Discount Depth (%)", nm, label_format = "percent")
})

(p_disc[[1]] | p_disc[[2]] | p_disc[[3]]) +
  plot_annotation(
    title = "Monthly Average Discount Depth on Steam",
    subtitle = "By game age window | dashed = Epic launch"
  ) &
  coord_cartesian(ylim = c(0, 32))
```

Monthly Average Discount Depth on Steam
By game age window | dashed = Epic launch



Section 1.2: Steam-Only vs Dual-Platform vs Future Dual-Platform

Restricts to the post-Epic-launch period and splits the Steam catalog into three groups: - Steam-Only: never joins Epic - Dual-Platform: currently listed on Epic (date >= entry_date) - Future Dual-Platform: will join Epic but has not yet (date < entry_date, entry_date not NA)

Same age windows as above. Includes 95% CI ribbons.

Helper: build post-launch panel with three groups

```
build_postlaunch_panel <- function(age_range, min_frac = 0.05) {
  panel <- data %>%
    filter(shop_name == "Steam") %>%
    mutate(days_since_release = as.numeric(date - release_date)) %>%
    filter(days_since_release >= age_range[1],
      days_since_release < age_range[2]) %>%
    filter(date >= epic_launch) %>%
    left_join(epic_entry_dates, by = "game_title") %>%
    mutate(
      group = case_when(
        is.na(entry_date) ~ "Steam-Only",
        date >= entry_date ~ "Dual-Platform",
        date < entry_date ~ "Future Dual-Platform",
        TRUE ~ NA_character_
      ),

```

```

    month = floor_date(date, "month")
  ) %>%
  filter(month < as.Date("2024-12-01")) %>%
  group_by(group, month) %>%
  summarise(
    avg_real_price = mean(real_price,      na.rm = TRUE),
    avg_sale_freq  = mean(sale,            na.rm = TRUE),
    avg_disc_depth = mean(discount_percent, na.rm = TRUE),
    n_games        = n_distinct(game_title),
    .groups        = "drop"
  )

# Drop sparse months WITHIN each group (per-group median, so each
# series is trimmed on its own scale rather than a pooled one)
panel %>%
  group_by(group) %>%
  filter(n_games >= median(n_games) * min_frac) %>%
  ungroup()
}

```

Helper: % of Dual/Future-DP games actually on Epic within window

```

pct_on_epic_in_window <- function(age_range) {
  data %>%
    filter(shop_name == "Steam") %>%
    distinct(game_title, release_date) %>%
    left_join(epic_entry_dates, by = "game_title") %>%
    filter(!is.na(entry_date)) %>%
    mutate(
      days_to_epic      = as.numeric(entry_date - release_date),
      on_epic_in_window = days_to_epic >= age_range[1] &
                           days_to_epic <  age_range[2]
    ) %>%
    summarise(pct = mean(on_epic_in_window, na.rm = TRUE) * 100) %>%
    pull(pct) %>%
    round(1)
}

```

Helper: three-group plot with 95% CI ribbons

```

plot_group_comparison <- function(df, outcome_col, y_lab,
                                   window_label, pct_on_epic) {
  ggplot(df, aes(x = month, y = .data[[outcome_col]], color = group, fill = group)) +
    geom_line(linewidth = 0.5, alpha = 0.5) +
    geom_smooth(method = "lm", se = TRUE, linewidth = 0.9, alpha = 0.15) +
    scale_color_manual(
      values = c("Steam-Only"           = "steelblue",
                  "Dual-Platform"       = "firebrick",
                  "Future Dual-Platform" = "darkorange"),
      name   = NULL
    ) +
    scale_fill_manual(

```

```

    values = c("Steam-Only"           = "steelblue",
               "Dual-Platform"       = "firebrick",
               "Future Dual-Platform" = "darkorange"),
    name    = NULL
  ) +
  labs(
    subtitle = paste0(window_label),
    x = NULL, y = y_lab
  ) +
  theme_fivethirtyeight() +
  theme(
    plot.subtitle = element_text(face = "bold", hjust = 0.5),
    plot.background = element_rect(fill = "white", color = NA),
    panel.background = element_rect(fill = "#F7F7F7", color = NA),
    legend.position = "bottom",
    legend.background = element_rect(fill = NA, color = NA),
    legend.key        = element_rect(fill = NA, color = NA)
  )
}

```

Build post-launch datasets

```

postlaunch_data <- map(graph_windows, build_postlaunch_panel)
postlaunch_pct  <- map(graph_windows, pct_on_epic_in_window)

```

Real Price Comparison

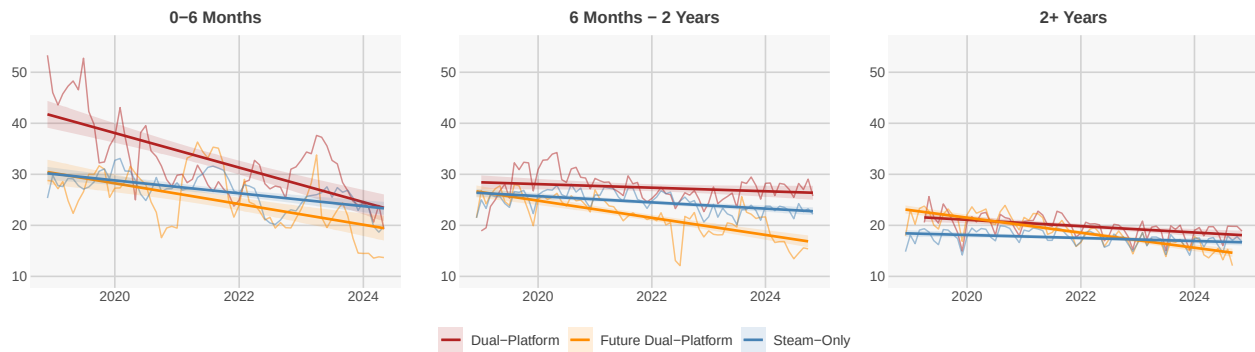
```

pl_price <- map2(names(postlaunch_data), postlaunch_data, function(nm, df) {
  plot_group_comparison(df, "avg_real_price", "Avg Real Price (USD)",
                        nm, postlaunch_pct[[nm]])
})

(pl_price[[1]] | pl_price[[2]] | pl_price[[3]]) +
  plot_annotation(
    title    = "Post-Launch Real Price on Steam",
    subtitle = "By Game Age Window | Trend lines with 95% CI"
  ) +
  plot_layout(guides = "collect") &
  theme(legend.position = "bottom") &
  coord_cartesian(ylim = c(10, 55))

```

Post-Launch Real Price on Steam
By Game Age Window | Trend lines with 95% CI

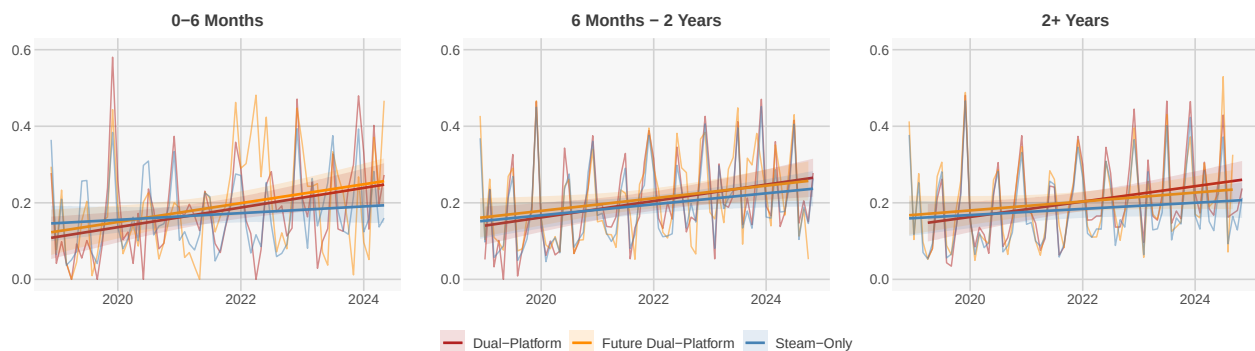


Sale Frequency Comparison

```
pl_sale <- map2(names(postlaunch_data), postlaunch_data, function(nm, df) {
  plot_group_comparison(df, "avg_sale_freq", "Avg Sale Frequency",
    nm, postlaunch_pct[[nm]])
})

(pl_sale[[1]] | pl_sale[[2]] | pl_sale[[3]]) +
  plot_annotation(
    title = "Post-Launch Sale Frequency on Steam",
    subtitle = "By Game Age Window | Trend lines with 95% CI"
  ) +
  plot_layout(guides = "collect") &
  theme(legend.position = "bottom") &
  coord_cartesian(ylim = c(0, .6))
```

Post-Launch Sale Frequency on Steam
By Game Age Window | Trend lines with 95% CI



Discount Depth Comparison

```
pl_disc <- map2(names(postlaunch_data), postlaunch_data, function(nm, df) {
  plot_group_comparison(df, "avg_disc_depth", "Avg Discount Depth (%)",
    nm, postlaunch_pct[[nm]])
})

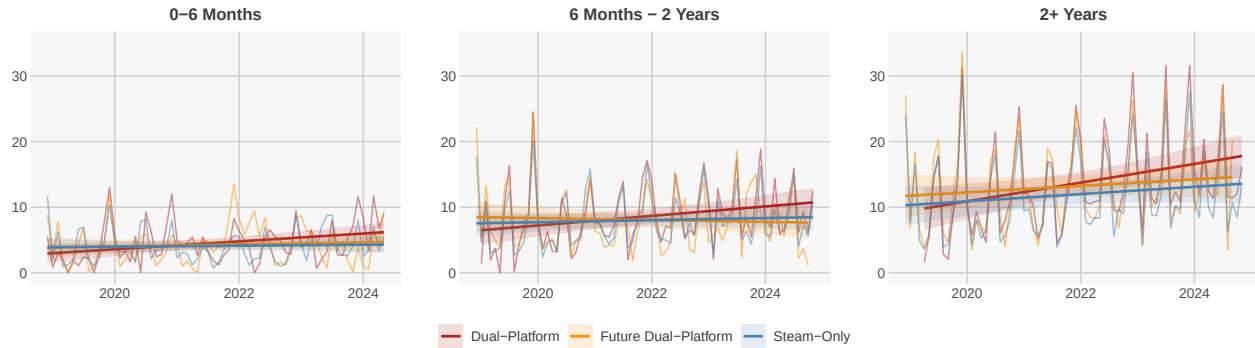
(pl_disc[[1]] | pl_disc[[2]] | pl_disc[[3]]) +
  plot_annotation(
```

```

title = "Post-Launch Discount Depth on Steam",
subtitle = "By Game Age Window | Trend lines with 95% CI"
) +
plot_layout(guides = "collect") &
theme(legend.position = "bottom") &
coord_cartesian(ylim = c(0, 35))

```

Post-Launch Discount Depth on Steam
By Game Age Window | Trend lines with 95% CI



Section 1.3: Regression Models

Two model families per outcome. Model 1 has no game FE with genre controls; Model 2 has game FE (genres absorbed).

Build Regression Dataset

```

reg_data <- data %>%
  filter(shop_name == "Steam") %>%
  mutate(month = floor_date(date, "month")) %>%
  group_by(game_title, month) %>%
  summarise(
    avg_real_price = mean(real_price, na.rm = TRUE),
    avg_price = mean(price, na.rm = TRUE),
    avg_sale_freq = mean(sale, na.rm = TRUE),
    avg_disc_depth = mean(discount_percent, na.rm = TRUE),
    release_date = first(release_date),
    across(all_of(genre_cols), first),
    .groups = "drop"
  ) %>%
  filter(year(month) > 2013) %>%
  left_join(epic_entry_dates, by = "game_title") %>%
  mutate(
    ever_on_epic = as.integer(!is.na(entry_date)),
    year = year(month),
    month_of_year = month(month),
    quarters_since_release = as.numeric(month - release_date) / (365.25 / 4),
    log_qsr = log(pmax(quarters_since_release, 0) + 1)
  )

```

Helper: run two models for one outcome

```
genre_string <- paste(genre_cols, collapse = " + ")

run_two_models <- function(outcome, tsr_var) {
  # Model 1: no game FE, with genre + ever_on_epic + year main effects + interactions
  f1 <- as.formula(paste0(
    outcome,
    " ~ i(year, ref = 2018) + ever_on_epic + i(year, ever_on_epic, ref = 2018) + ",
    tsr_var, " + ", genre_string,
    " | month_of_year"
  ))
  m1 <- feols(f1, data = reg_data, cluster = ~ game_title)

  # Model 2: game FE (absorbs ever_on_epic and genres)
  f2 <- as.formula(paste0(
    outcome,
    " ~ i(year, ref = 2018) + i(year, ever_on_epic, ref = 2018) + ",
    tsr_var,
    " | game_title + month_of_year"
  ))
  m2 <- feols(f2, data = reg_data, cluster = ~ game_title)

  list(m1 = m1, m2 = m2)
}
```

Nominal Price

```
price_models <- run_two_models("avg_price", "log_qsr")

etable(price_models$m1, price_models$m2,
  headers = c("No Game FE (Model 1)", "Game FE (Model 2)"),
  title = "Nominal Price - Full regression output (for review)")
```

	price_models\$m1	price_models\$m2
	No Game FE (Model 1)	Game FE (Model 2)
Dependent Var.:	avg_price	avg_price
year = 2014	-8.779*** (1.482)	0.1231 (1.057)
year = 2015	-5.566*** (1.075)	1.121 (0.8163)
year = 2016	-3.631*** (0.7302)	0.4803 (0.5396)
year = 2017	-1.781*** (0.3895)	0.0158 (0.2783)
year = 2019	1.512*** (0.3150)	-0.1165 (0.2157)
year = 2020	2.808*** (0.4775)	-0.3018 (0.3129)
year = 2021	3.499*** (0.6062)	-0.6651. (0.3962)
year = 2022	4.160*** (0.6701)	-0.9661* (0.4382)
year = 2023	4.946*** (0.7082)	-1.224** (0.4661)
year = 2024	5.969*** (0.7620)	-1.212* (0.5173)
ever_on_epic	3.478** (1.082)	
ever_on_epic x year = 2014	5.886. (3.032)	5.190* (2.603)
ever_on_epic x year = 2015	4.972* (2.026)	3.699* (1.756)
ever_on_epic x year = 2016	2.509* (1.273)	1.643 (1.009)
ever_on_epic x year = 2017	1.111 (0.7085)	0.8579 (0.5532)
ever_on_epic x year = 2019	-0.4188 (0.5039)	-0.4623 (0.3408)

```

## ever_on_epic x year = 2020      -1.012 (0.7365)   -0.7866 (0.5080)
## ever_on_epic x year = 2021      -0.7453 (0.8750)   -0.7261 (0.5985)
## ever_on_epic x year = 2022      -1.224 (0.9702)   -1.506* (0.6860)
## ever_on_epic x year = 2023      -0.6723 (0.9999)   -1.657* (0.7090)
## ever_on_epic x year = 2024      -0.8368 (1.010)   -1.922** (0.7139)
## log_qsr                         -4.705*** (0.3554) -1.219*** (0.2521)
## RPG                             2.734** (1.038)
## Action                           1.975* (0.9131)
## Adventure                        -1.804* (0.9171)
## Shooter                           0.7148 (1.633)
## Strategy                          1.768 (1.157)
## Simulation                       -3.494** (1.193)
## Platformer                       -3.493** (1.059)
## Puzzle                           -4.982*** (1.244)
## Horror                           -1.165 (1.251)
## OpenWorld                        4.852*** (1.000)
## Survival                         -2.694* (1.163)
## Roguelike                        -9.171*** (0.9705)
## Multiplayer                       -0.4452 (1.008)
## Narrative                         0.5993 (1.095)
## Fixed-Effects:
## month_of_year                     Yes           Yes
## game_title                        No           Yes
## -----
## S.E.: Clustered                   by: game_title   by: game_title
## Observations                      69,654         69,654
## R2                                0.21742        0.84334
## Within R2                          0.20906        0.12810
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

Real Price

```
realprice_models <- run_two_models("avg_real_price", "log_qsr")
```

```

etable(realprice_models$m1, realprice_models$m2,
  headers = c("No Game FE (Model 1)", "Game FE (Model 2)"),
  title   = "Real Price - Full regression output (for review)")

```

```

##                      realprice_model..1 realprice_model..2
##                      No Game FE (Model 1) Game FE (Model 2)
## Dependent Var.:      avg_real_price      avg_real_price
##
## year = 2014           -6.410*** (1.452)      0.0830 (1.056)
## year = 2015           -3.622*** (1.053)      1.294 (0.8179)
## year = 2016           -2.265** (0.7105)      0.7699 (0.5369)
## year = 2017           -1.075** (0.3735)      0.2347 (0.2722)
## year = 2019           0.9494** (0.2931)     -0.2076 (0.2066)
## year = 2020           1.800*** (0.4420)     -0.3933 (0.3022)
## year = 2021           1.603** (0.5557)    -1.291*** (0.3830)
## year = 2022           0.7421 (0.6061)    -2.758*** (0.4246)
## year = 2023           0.6053 (0.6365)    -3.544*** (0.4576)
## year = 2024           0.9520 (0.6796)    -3.849*** (0.5079)
## ever_on_epic          3.346** (1.024)

```

```

## ever_on_epic x year = 2014      6.033* (3.022)      5.426* (2.590)
## ever_on_epic x year = 2015      5.149* (2.020)      4.092* (1.763)
## ever_on_epic x year = 2016      2.691* (1.260)      1.945. (1.014)
## ever_on_epic x year = 2017      1.214. (0.6842)      0.9529. (0.5446)
## ever_on_epic x year = 2019     -0.4807 (0.4735)     -0.4835 (0.3283)
## ever_on_epic x year = 2020     -1.043 (0.6906)    -0.8351. (0.4900)
## ever_on_epic x year = 2021     -0.9506 (0.8215)    -0.8967 (0.5850)
## ever_on_epic x year = 2022     -1.570. (0.9020)    -1.721* (0.6670)
## ever_on_epic x year = 2023     -1.249 (0.9237)    -1.904** (0.6920)
## ever_on_epic x year = 2024     -1.437 (0.9327)    -2.144** (0.7029)
## log_qsr                        -4.025*** (0.3104) -1.682*** (0.2469)
## RPG                           2.233* (0.9162)
## Action                        1.629* (0.8056)
## Adventure                     -1.690* (0.8056)
## Shooter                       0.5350 (1.442)
## Strategy                      1.479 (1.024)
## Simulation                    -3.130** (1.053)
## Platformer                   -3.160*** (0.9280)
## Puzzle                       -4.440*** (1.112)
## Horror                       -1.108 (1.107)
## OpenWorld                    4.255*** (0.8776)
## Survival                     -2.453* (1.026)
## Roguelike                    -8.090*** (0.8526)
## Multiplayer                  -0.4318 (0.8916)
## Narrative                     0.4546 (0.9610)
## Fixed-Effects:
## month_of_year                  Yes                Yes
## game_title                     No                Yes
## -----
## S.E.: Clustered                by: game_title    by: game_title
## Observations                   69,654          69,654
## R2                            0.22947          0.82396
## Within R2                     0.22120          0.30866
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

Sale Frequency

```

sale_models <- run_two_models("avg_sale_freq", "log_qsr")

etable(sale_models$m1, sale_models$m2,
  headers = c("No Game FE (Model 1)", "Game FE (Model 2)"),
  title   = "Sale Frequency - Full regression output (for review)")

```

```

##                sale_models$m1    sale_models$m2
##                No Game FE (Model 1)  Game FE (Model 2)
## Dependent Var.:                avg_sale_freq                avg_sale_freq
##
## year = 2014                0.0369* (0.0177)    0.0541** (0.0188)
## year = 2015                -0.0075 (0.0082)     0.0037 (0.0102)
## year = 2016                -0.0108* (0.0044)    -0.0058 (0.0057)
## year = 2017                -0.0112** (0.0041)   -0.0081. (0.0044)
## year = 2019                0.0241*** (0.0042)   0.0208*** (0.0045)
## year = 2020                0.0180*** (0.0047)   0.0108. (0.0056)

```

```

## year = 2021          0.0242*** (0.0045)    0.0155* (0.0063)
## year = 2022          0.0496*** (0.0055)  0.0381*** (0.0077)
## year = 2023          0.0731*** (0.0058)  0.0588*** (0.0087)
## year = 2024          0.0830*** (0.0055)  0.0660*** (0.0094)
## ever_on_epic         0.0149* (0.0069)
## ever_on_epic x year = 2014 -0.0577* (0.0236) -0.0443. (0.0256)
## ever_on_epic x year = 2015 -0.0271* (0.0133) -0.0178 (0.0144)
## ever_on_epic x year = 2016 -0.0099 (0.0089) -0.0010 (0.0092)
## ever_on_epic x year = 2017 -0.0001 (0.0086)  0.0037 (0.0087)
## ever_on_epic x year = 2019 -0.0037 (0.0068) -0.0034 (0.0070)
## ever_on_epic x year = 2020 -0.0023 (0.0080) -0.0028 (0.0081)
## ever_on_epic x year = 2021  0.0019 (0.0081)  0.0004 (0.0083)
## ever_on_epic x year = 2022  0.0159. (0.0092)  0.0156 (0.0099)
## ever_on_epic x year = 2023  0.0111 (0.0094)  0.0123 (0.0106)
## ever_on_epic x year = 2024  0.0211* (0.0087)  0.0210* (0.0101)
## log_qsr              0.0012 (0.0023)  0.0077. (0.0040)
## RPG                  -0.0054 (0.0049)
## Action               0.0139** (0.0051)
## Adventure            0.0122* (0.0058)
## Shooter              0.0078 (0.0066)
## Strategy             0.0211** (0.0068)
## Simulation           0.0061 (0.0067)
## Platformer           0.0063 (0.0077)
## Puzzle               0.0128 (0.0098)
## Horror               0.0115 (0.0071)
## OpenWorld            0.0010 (0.0060)
## Survival             -0.0060 (0.0074)
## Roguelike            0.0123 (0.0080)
## Multiplayer          0.0204*** (0.0060)
## Narrative            0.0092. (0.0056)
## Fixed-Effects:      -----
## month_of_year                Yes                Yes
## game_title                   No                Yes
## -----
## S.E.: Clustered      by: game_title      by: game_title
## Observations                69,654                69,654
## R2                        0.26121                0.35304
## Within R2                 0.05443                0.03737
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

Discount Depth

```

disc_models <- run_two_models("avg_disc_depth", "log_qsr")

etable(disc_models$m1, disc_models$m2,
  headers = c("No Game FE (Model 1)", "Game FE (Model 2)"),
  title   = "Discount Depth - Full regression output (for review)")

```

```

##                disc_models$m1    disc_models$m2
##                No Game FE (Model 1) Game FE (Model 2)
## Dependent Var.:      avg_disc_depth      avg_disc_depth
##
## year = 2014          2.738*** (0.5661)  2.557** (0.8027)

```

```

## year = 2015          1.368** (0.4210)    1.384* (0.6033)
## year = 2016          0.7064** (0.2537)   0.7420* (0.3542)
## year = 2017          -0.1015 (0.1933)   -0.0722 (0.2218)
## year = 2019          0.9898*** (0.2693)  0.9115** (0.2970)
## year = 2020          0.2288 (0.2951)    0.0807 (0.3511)
## year = 2021          0.3146 (0.2913)    0.2306 (0.4043)
## year = 2022          1.479*** (0.3572)   1.468** (0.5081)
## year = 2023          2.453*** (0.3994)   2.558*** (0.5925)
## year = 2024          2.745*** (0.4017)   2.910*** (0.6331)
## ever_on_epic         1.002** (0.3779)
## ever_on_epic x year = 2014 -2.401** (0.7589)    -1.163 (1.042)
## ever_on_epic x year = 2015 -1.385* (0.5865)    -0.6086 (0.7732)
## ever_on_epic x year = 2016 -0.8466* (0.4010)   -0.2397 (0.5173)
## ever_on_epic x year = 2017 -0.4833. (0.2918)   -0.2657 (0.3269)
## ever_on_epic x year = 2019  0.3805 (0.3668)    0.4665 (0.3937)
## ever_on_epic x year = 2020  0.0360 (0.4163)    0.3187 (0.4717)
## ever_on_epic x year = 2021  0.2837 (0.4347)    0.7446 (0.5033)
## ever_on_epic x year = 2022  1.317* (0.5610)    2.026** (0.6770)
## ever_on_epic x year = 2023  1.215. (0.6207)    2.316** (0.7826)
## ever_on_epic x year = 2024  2.175*** (0.5799)  3.408*** (0.7557)
## log_qsr              3.071*** (0.1510)   2.994*** (0.2073)
## RPG                  -0.0481 (0.3983)
## Action               2.004*** (0.3910)
## Adventure            1.196** (0.4320)
## Shooter              -0.2210 (0.4676)
## Strategy             1.854*** (0.5574)
## Simulation           -0.2136 (0.6063)
## Platformer           -0.2310 (0.6005)
## Puzzle               1.118 (0.8949)
## Horror               1.235* (0.5600)
## OpenWorld            0.6794 (0.4693)
## Survival             -1.047. (0.6300)
## Roguelike            0.0620 (0.6484)
## Multiplayer          1.085* (0.4796)
## Narrative            0.1933 (0.4394)
## Fixed-Effects:      -----
## month_of_year                Yes                Yes
## game_title                   No                Yes
## -----
## S.E.: Clustered          by: game_title    by: game_title
## Observations              69,654          69,654
## R2                        0.28841          0.42057
## Within R2                 0.13022          0.12785
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

Section 1.4: Thesis Display Output

I needed to make Word output of the models for the thesis.

```
omit_regex <- paste0("Intercept|^",
  paste(genre_cols, collapse = "$|^"),
```

```

"$")

coef_rename <- c(
  "ever_on_epic"      = "Ever on Epic",
  "log_qsr"           = "Log(Quarters Since Release)",
  "quarters_since_release" = "Quarters Since Release"
)

# Game FE models only (m2), one column per outcome
s1_models <- list(
  "Nominal Price (USD)" = price_models$m2,
  "Real Price (USD)"    = realprice_models$m2,
  "Sale Freq. (share)"  = sale_models$m2,
  "Discount Depth (pp)" = disc_models$m2
)

# Build coef_map programmatically so year effects group first, then the
# interactions, then the trend. Years derived from the data (length-safe),
# 2018 dropped as the reference.
years      <- sort(setdiff(unique(reg_data$year), 2018))
year_terms <- paste0("year::", years) # main effects
int_terms  <- paste0("year::", years, ":ever_on_epic") # interactions

coef_map <- c(
  setNames(as.character(years), year_terms),
  setNames(paste0(years, " x Epic"), int_terms),
  "log_qsr" = "Log(QSR)"
)

s1_ft <- modelsummary(
  s1_models,
  coef_map = coef_map,
  gof_omit = "^R2$|R2 Adj\\.|R2 Within Adj\\.|AIC|BIC|Log\\.Lik\\.|RMSE|Std\\.Errors|FE:",
  stars    = c('*' = .05, '**' = .01, '***' = .001),
  title    = "Year-Relative Effects on Steam Outcomes",
  notes    = c(
    "All models include game and month-of-year fixed effects. Reference year 2018. SEs clustered by game"
  ),
  output   = "flextable"
) %>%
  flextable::autofit()

s1_ft

```

```

## Warning: fonts used in `flextable` are ignored because the `pdflatex` engine is
## used and not `xelatex` or `lualatex`. You can avoid this warning by using
## `set_flextable_defaults(fonts_ignore=TRUE)` or use a compatible engine (e.g.
## `pdf-engine: xelatex`).

```

Table 1: Year-Relative Effects on Steam Outcomes

	Nominal Price (USD)	Real Price (USD)	Sale Freq. (share)	Discount Depth (pp)
2014	0.123	0.083	0.054**	2.557**

Table 1: Year-Relative Effects on Steam Outcomes

	Nominal Price (USD)	Real Price (USD)	Sale Freq. (share)	Discount Depth (pp)
	(1.057)	(1.056)	(0.019)	(0.803)
2015	1.121	1.294	0.004	1.384*
	(0.816)	(0.818)	(0.010)	(0.603)
2016	0.480	0.770	-0.006	0.742*
	(0.540)	(0.537)	(0.006)	(0.354)
2017	0.016	0.235	-0.008	-0.072
	(0.278)	(0.272)	(0.004)	(0.222)
2019	-0.116	-0.208	0.021***	0.912**
	(0.216)	(0.207)	(0.005)	(0.297)
2020	-0.302	-0.393	0.011	0.081
	(0.313)	(0.302)	(0.006)	(0.351)
2021	-0.665	-1.291***	0.016*	0.231
	(0.396)	(0.383)	(0.006)	(0.404)
2022	-0.966*	-2.758***	0.038***	1.468**
	(0.438)	(0.425)	(0.008)	(0.508)
2023	-1.224**	-3.544***	0.059***	2.558***
	(0.466)	(0.458)	(0.009)	(0.593)
2024	-1.212*	-3.849***	0.066***	2.910***
	(0.517)	(0.508)	(0.009)	(0.633)
2014 x Epic	5.190*	5.426*	-0.044	-1.163
	(2.603)	(2.590)	(0.026)	(1.042)
2015 x Epic	3.699*	4.092*	-0.018	-0.609
	(1.756)	(1.763)	(0.014)	(0.773)
2016 x Epic	1.643	1.945	-0.001	-0.240
	(1.009)	(1.014)	(0.009)	(0.517)
2017 x Epic	0.858	0.953	0.004	-0.266
	(0.553)	(0.545)	(0.009)	(0.327)
2019 x Epic	-0.462	-0.483	-0.003	0.466
	(0.341)	(0.328)	(0.007)	(0.394)
2020 x Epic	-0.787	-0.835	-0.003	0.319
	(0.508)	(0.490)	(0.008)	(0.472)
2021 x Epic	-0.726	-0.897	0.000	0.745
	(0.598)	(0.585)	(0.008)	(0.503)
2022 x Epic	-1.506*	-1.721*	0.016	2.026**

Table 1: Year-Relative Effects on Steam Outcomes

	Nominal Price (USD)	Real Price (USD)	Sale Freq. (share)	Discount Depth (pp)
	(0.686)	(0.667)	(0.010)	(0.677)
2023 x Epic	-1.657*	-1.904**	0.012	2.316**
	(0.709)	(0.692)	(0.011)	(0.783)
2024 x Epic	-1.922**	-2.144**	0.021*	3.408***
	(0.714)	(0.703)	(0.010)	(0.756)
Log(QSR)	-1.219***	-1.682***	0.008	2.994***
	(0.252)	(0.247)	(0.004)	(0.207)
Num.Obs.	69654	69654	69654	69654
R2 Within	0.128	0.309	0.037	0.128

* $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$

All models include game and month-of-year fixed effects. Reference year 2018. SEs clustered by game

```
# save table
save_as_docx(s1_ft, path = "Output/tab_section1_regs.docx")
```

Section 1.5: Year-Relative Trajectory Plots

Extracts the year main effects and their interactions from the regression models to plot Steam-Only vs Dual-Platform trajectories relative to 2018.

Helper: extract trajectories with 95% CI

```
extract_trajectories <- function(model, model_label, outcome_label) {
  V <- vcov(model)
  co <- coef(model)

  year_terms <- grep("^year::[0-9]+$", names(co), value = TRUE)
  years <- as.numeric(sub("year::", "", year_terms))

  map_dfr(seq_along(year_terms), function(i) {
    y_term <- year_terms[i]
    int_term <- paste0(y_term, ":ever_on_epic")

    so_est <- co[y_term]
    so_se <- sqrt(V[y_term, y_term])

    if (int_term %in% names(co)) {
      dp_est <- co[y_term] + co[int_term]
      dp_var <- V[y_term, y_term] + V[int_term, int_term] +
        2 * V[y_term, int_term]
      dp_se <- sqrt(dp_var)
    } else {
```

```

    dp_est <- NA_real_
    dp_se  <- NA_real_
  }

  tibble(
    model      = model_label,
    outcome    = outcome_label,
    year       = years[i],
    steam_only = so_est,
    so_se      = so_se,
    dual_plat  = dp_est,
    dp_se      = dp_se
  )
})
}

```

Build long-form plot dataset

```

plot_data <- bind_rows(
  # extract_trajectories(price_models$m1,      "No Game FE", "Nominal Price"),
  extract_trajectories(price_models$m2,      "Game FE",    "Nominal Price"),
  # extract_trajectories(realprice_models$m1, "No Game FE", "Real Price"),
  extract_trajectories(realprice_models$m2, "Game FE",    "Real Price"),
  # extract_trajectories(sale_models$m1,      "No Game FE", "Sale Frequency"),
  extract_trajectories(sale_models$m2,      "Game FE",    "Sale Frequency"),
  # extract_trajectories(disc_models$m1,      "No Game FE", "Discount Depth"),
  extract_trajectories(disc_models$m2,      "Game FE",    "Discount Depth")
)

# Add reference-year (2018) rows at zero for visual continuity
ref_year_rows <- plot_data %>%
  distinct(model, outcome) %>%
  mutate(year = 2018, steam_only = 0, so_se = 0,
         dual_plat = 0, dp_se = 0)

plot_data <- bind_rows(plot_data, ref_year_rows) %>%
  arrange(model, outcome, year)

# Reshape to long form for plotting
plot_long <- plot_data %>%
  pivot_longer(cols = c(steam_only, dual_plat),
               names_to = "group", values_to = "estimate") %>%
  mutate(
    se      = if_else(group == "steam_only", so_se, dp_se),
    group   = recode(group,
                     steam_only = "Steam-Only",
                     dual_plat  = "Dual-Platform"),
    ci_lower = estimate - 1.96 * se,
    ci_upper = estimate + 1.96 * se
  ) %>%
  select(model, outcome, year, group, estimate, ci_lower, ci_upper)

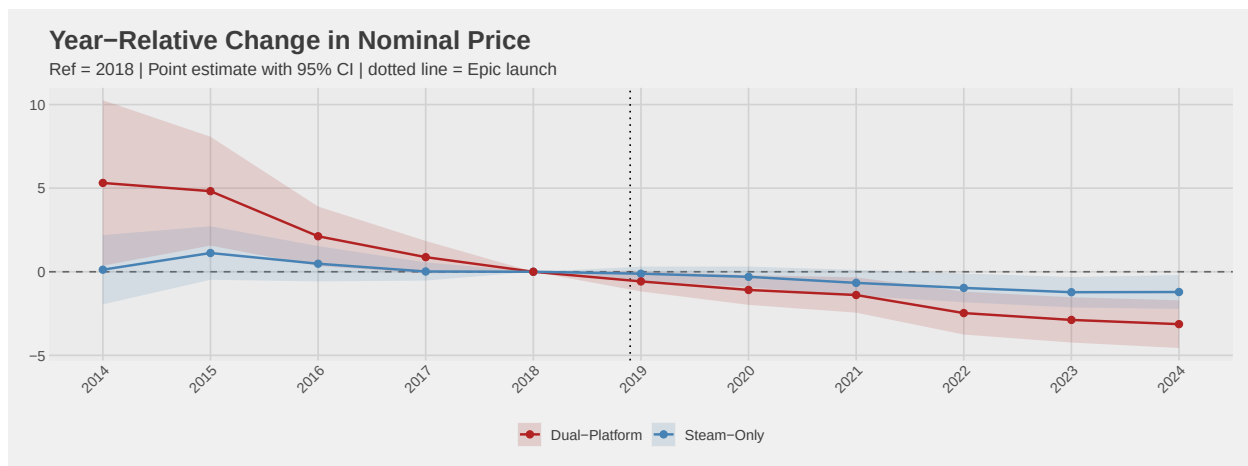
```

Plot helper (faceted by model, one outcome per plot)

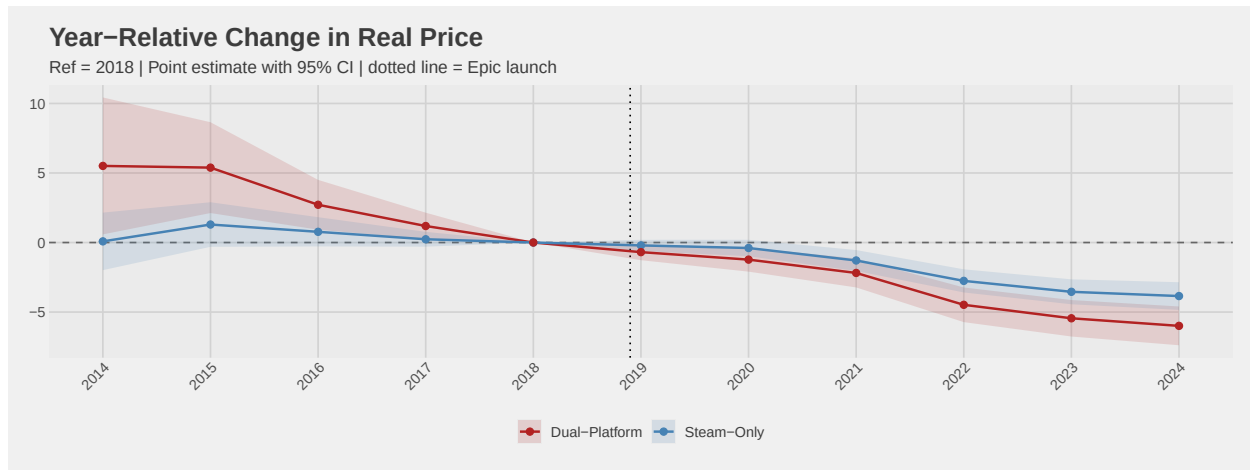
```
plot_trajectories <- function(df, outcome_label) {
  df %>%
    filter(outcome == outcome_label) %>%
    ggplot(aes(x = year, y = estimate, color = group, fill = group)) +
    geom_hline(yintercept = 0, linetype = "dashed", color = "gray40") +
    geom_vline(xintercept = 2018.9,
               linetype = "dotted", color = "black", alpha = 1) +
    geom_ribbon(aes(ymin = ci_lower, ymax = ci_upper),
               alpha = 0.15, color = NA) +
    geom_line(linewidth = 0.8) +
    geom_point(size = 2) +
    # facet_wrap(~ model, nrow = 1) +
    scale_color_manual(values = c("Steam-Only" = "steelblue",
                                   "Dual-Platform" = "firebrick"),
                       name = NULL) +
    scale_fill_manual(values = c("Steam-Only" = "steelblue",
                                   "Dual-Platform" = "firebrick"),
                       name = NULL) +
    scale_x_continuous(breaks = unique(df$year)) +
    labs(
      title = paste0("Year-Relative Change in ", outcome_label),
      subtitle = "Ref = 2018 | Point estimate with 95% CI | dotted line = Epic launch",
      x = "Year",
      y = paste0("Change from 2018 (", outcome_label, ")")
    ) +
    theme_fivethirtyeight() +
    theme(legend.position = "bottom",
          axis.text.x = element_text(angle = 45, hjust = 1))
}
```

Generate one plot per outcome

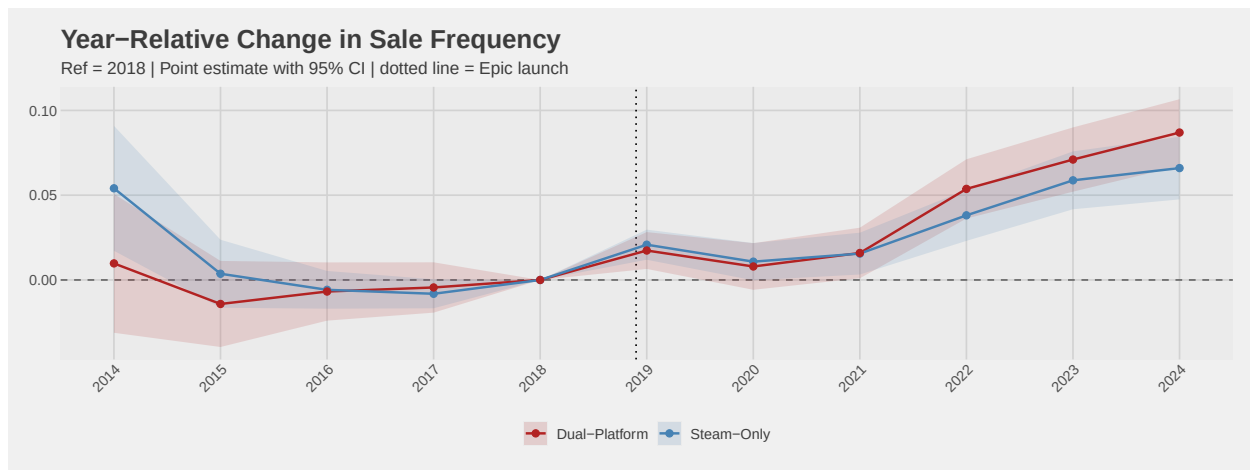
```
plot_trajectories(plot_long, "Nominal Price")
```



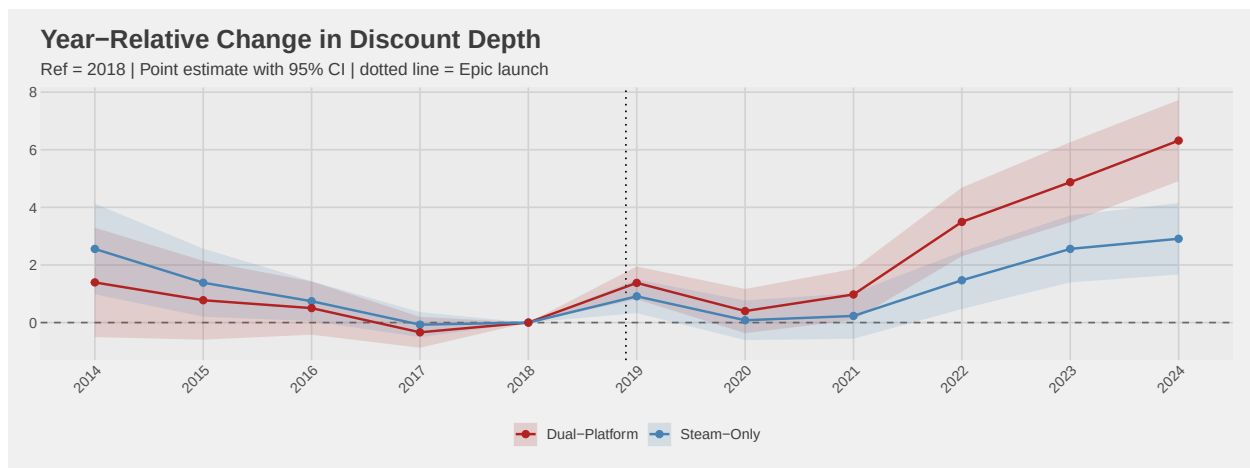
```
plot_trajectories(plot_long, "Real Price")
```



```
plot_trajectories(plot_long, "Sale Frequency")
```



```
plot_trajectories(plot_long, "Discount Depth")
```



Extra Information and Code

Sanity checks on regression data

```
cat("Observations by year:\n")

## Observations by year:
reg_data %>% count(year) %>% print()

## # A tibble: 11 x 2
##   year      n
##   <dbl> <int>
## 1  2014  1014
## 2  2015  2209
## 3  2016  3385
## 4  2017  4583
## 5  2018  5680
## 6  2019  6538
## 7  2020  7435
## 8  2021  8423
## 9  2022  9272
## 10 2023 10348
## 11 2024 10767

cat("\nEver on Epic counts:\n")

##
## Ever on Epic counts:
reg_data %>%
  distinct(game_title, ever_on_epic) %>%
  count(ever_on_epic) %>% print()

## # A tibble: 2 x 2
##   ever_on_epic      n
##   <int> <int>
## 1         0    544
## 2         1    356

cat("\nMonth-of-year distribution:\n")

##
## Month-of-year distribution:
reg_data %>% count(month_of_year) %>% print()

## # A tibble: 12 x 2
##   month_of_year      n
##   <dbl> <int>
## 1         1    5397
## 2         2    5465
## 3         3    5537
## 4         4    5602
## 5         5    5679
## 6         6    5768
## 7         7    5832
## 8         8    5919
```

```
## 9          9 6010
## 10         10 6084
## 11         11 6150
## 12         12 6211
```

Full Regression Output (No Game FE vs Game FE)

Nominal Price — Thesis Table

```
modelsummary(
  list("No Game FE" = price_models$m1, "Game FE" = price_models$m2),
  stars      = TRUE,
  coef_omit  = omit_regex,
  coef_rename = coef_rename,
  gof_omit   = "AIC|BIC|Log.Lik|F|RMSE",
  title      = "Nominal Price: Year-Relative Effects",
  notes      = "Reference year 2018. Year main effects show market-wide change relative to 2018; year :
  output = "flextable"
)
```

Table 2: Nominal Price: Year-Relative Effects

	No Game FE	Game FE
year::2014	-8.779*** (1.482)	0.123 (1.057)
year::2015	-5.566*** (1.075)	1.121 (0.816)
year::2016	-3.631*** (0.730)	0.480 (0.540)
year::2017	-1.781*** (0.389)	0.016 (0.278)
year::2019	1.512*** (0.315)	-0.116 (0.216)
year::2020	2.808*** (0.478)	-0.302 (0.313)
year::2021	3.499*** (0.606)	-0.665+ (0.396)
year::2022	4.160*** (0.670)	-0.966* (0.438)
year::2023	4.946*** (0.708)	-1.224** (0.466)
year::2024	5.969*** (0.762)	-1.212* (0.517)

Table 2: Nominal Price: Year-Relative Effects

	No Game FE	Game FE
Ever on Epic	3.478** (1.082)	
year::2014:Ever on Epic	5.886+ (3.032)	5.190* (2.603)
year::2015:Ever on Epic	4.972* (2.026)	3.699* (1.756)
year::2016:Ever on Epic	2.509* (1.273)	1.643 (1.009)
year::2017:Ever on Epic	1.111 (0.708)	0.858 (0.553)
year::2019:Ever on Epic	-0.419 (0.504)	-0.462 (0.341)
year::2020:Ever on Epic	-1.012 (0.737)	-0.787 (0.508)
year::2021:Ever on Epic	-0.745 (0.875)	-0.726 (0.598)
year::2022:Ever on Epic	-1.224 (0.970)	-1.506* (0.686)
year::2023:Ever on Epic	-0.672 (1.000)	-1.657* (0.709)
year::2024:Ever on Epic	-0.837 (1.010)	-1.922** (0.714)
Log(Quarters Since Release)	-4.705*** (0.355)	-1.219*** (0.252)

Table 2: Nominal Price: Year-Relative Effects

	No Game FE	Game FE
Num.Obs.	69654	69654
R2	0.217	0.843
R2 Adj.	0.217	0.841
R2 Within	0.209	0.128
R2 Within Adj.	0.209	0.128

+ p < 0.1, * p < 0.05, ** p < 0.01,
*** p < 0.001

Reference year 2018. Year main effects show market-wide change relative to 2018; year x ever_on_epic interactions show additional differential effect on dual-platform games.

Real Price — Thesis Table

```
modelsummary(
  list("No Game FE" = realprice_models$m1, "Game FE" = realprice_models$m2),
  stars = TRUE,
  coef_omit = omit_regex,
  coef_rename = coef_rename,
  gof_omit = "AIC|BIC|Log.Lik|F|RMSE",
  title = "Real Price: Year-Relative Effects",
  notes = "Reference year 2018.",
  output = "flextable"
)
```

Table 3: Real Price: Year-Relative Effects

	No Game FE	Game FE
year::2014	-6.410*** (1.452)	0.083 (1.056)
year::2015	-3.622*** (1.053)	1.294 (0.818)
year::2016	-2.265** (0.711)	0.770 (0.537)
year::2017	-1.075** (0.374)	0.235 (0.272)

Table 3: Real Price: Year-Relative Effects

	No Game FE	Game FE
year::2019	0.949** (0.293)	-0.208 (0.207)
year::2020	1.800*** (0.442)	-0.393 (0.302)
year::2021	1.603** (0.556)	-1.291*** (0.383)
year::2022	0.742 (0.606)	-2.758*** (0.425)
year::2023	0.605 (0.636)	-3.544*** (0.458)
year::2024	0.952 (0.680)	-3.849*** (0.508)
Ever on Epic	3.346** (1.024)	
year::2014:Ever on Epic	6.033* (3.022)	5.426* (2.590)
year::2015:Ever on Epic	5.149* (2.020)	4.092* (1.763)
year::2016:Ever on Epic	2.691* (1.260)	1.945+ (1.014)
year::2017:Ever on Epic	1.214+ (0.684)	0.953+ (0.545)
year::2019:Ever on Epic	-0.481 (0.474)	-0.483 (0.328)
year::2020:Ever on Epic	-1.043 (0.691)	-0.835+ (0.490)
year::2021:Ever on Epic	-0.951 (0.822)	-0.897 (0.585)

Table 3: Real Price: Year-Relative Effects

	No Game FE	Game FE
year::2022:Ever on Epic	-1.570+	-1.721*
	(0.902)	(0.667)
year::2023:Ever on Epic	-1.249	-1.904**
	(0.924)	(0.692)
year::2024:Ever on Epic	-1.437	-2.144**
	(0.933)	(0.703)
Log(Quarters Since Release)	-4.025***	-1.682***
	(0.310)	(0.247)
Num.Obs.	69654	69654
R2	0.229	0.824
R2 Adj.	0.229	0.822
R2 Within	0.221	0.309
R2 Within Adj.	0.221	0.308
+ p < 0.1, * p < 0.05, ** p < 0.01, *** p < 0.001		

Reference year 2018.

Sale Frequency — Thesis Table

```

modelsummary(
  list("No Game FE" = sale_models$m1, "Game FE" = sale_models$m2),
  stars      = TRUE,
  coef_omit  = omit_regex,
  coef_rename = coef_rename,
  gof_omit   = "AIC|BIC|Log.Lik|F|RMSE",
  title      = "Sale Frequency: Year-Relative Effects",
  notes      = "Reference year 2018.",
  output     = "flextable"
)

```

Table 4: Sale Frequency: Year-Relative Effects

	No Game FE	Game FE
year::2014	0.037* (0.018)	0.054** (0.019)
year::2015	-0.008 (0.008)	0.004 (0.010)
year::2016	-0.011* (0.004)	-0.006 (0.006)
year::2017	-0.011** (0.004)	-0.008+ (0.004)
year::2019	0.024*** (0.004)	0.021*** (0.005)
year::2020	0.018*** (0.005)	0.011+ (0.006)
year::2021	0.024*** (0.005)	0.016* (0.006)
year::2022	0.050*** (0.006)	0.038*** (0.008)
year::2023	0.073*** (0.006)	0.059*** (0.009)
year::2024	0.083*** (0.006)	0.066*** (0.009)
Ever on Epic	0.015* (0.007)	
year::2014:Ever on Epic	-0.058* (0.024)	-0.044+ (0.026)
year::2015:Ever on Epic	-0.027* (0.013)	-0.018 (0.014)
year::2016:Ever on Epic	-0.010 (0.009)	-0.001 (0.009)
year::2017:Ever on Epic	-0.000	0.004

Table 4: Sale Frequency: Year-Relative Effects

	No Game FE	Game FE
	(0.009)	(0.009)
year::2019:Ever on Epic	-0.004	-0.003
	(0.007)	(0.007)
year::2020:Ever on Epic	-0.002	-0.003
	(0.008)	(0.008)
year::2021:Ever on Epic	0.002	0.000
	(0.008)	(0.008)
year::2022:Ever on Epic	0.016+	0.016
	(0.009)	(0.010)
year::2023:Ever on Epic	0.011	0.012
	(0.009)	(0.011)
year::2024:Ever on Epic	0.021*	0.021*
	(0.009)	(0.010)
Log(Quarters Since Release)	0.001	0.008+
	(0.002)	(0.004)
Num.Obs.	69654	69654
R2	0.261	0.353
R2 Adj.	0.261	0.344
R2 Within	0.054	0.037
R2 Within Adj.	0.054	0.037
+ p < 0.1, * p < 0.05, ** p < 0.01, *** p < 0.001		

Reference year 2018.

Discount Depth — Thesis Table

```

modelsummary(
  list("No Game FE" = disc_models$m1, "Game FE" = disc_models$m2),
  stars      = TRUE,

```

```

coef_omit = omit_regex,
coef_rename = coef_rename,
gof_omit = "AIC|BIC|Log.Lik|F|RMSE",
title = "Discount Depth: Year-Relative Effects",
notes = "Reference year 2018.",
output = "flextable"
)

```

Table 5: Discount Depth: Year-Relative Effects

	No Game FE	Game FE
year::2014	2.738*** (0.566)	2.557** (0.803)
year::2015	1.368** (0.421)	1.384* (0.603)
year::2016	0.706** (0.254)	0.742* (0.354)
year::2017	-0.101 (0.193)	-0.072 (0.222)
year::2019	0.990*** (0.269)	0.912** (0.297)
year::2020	0.229 (0.295)	0.081 (0.351)
year::2021	0.315 (0.291)	0.231 (0.404)
year::2022	1.479*** (0.357)	1.468** (0.508)
year::2023	2.453*** (0.399)	2.558*** (0.593)
year::2024	2.745*** (0.402)	2.910*** (0.633)
Ever on Epic	1.002** (0.378)	
year::2014:Ever on Epic	-2.401** (0.759)	-1.163 (1.042)
year::2015:Ever on Epic	-1.385* (0.759)	-0.609 (1.042)

Table 5: Discount Depth: Year-Relative Effects

	No Game FE	Game FE
	(0.586)	(0.773)
year::2016:Ever on Epic	-0.847*	-0.240
	(0.401)	(0.517)
year::2017:Ever on Epic	-0.483+	-0.266
	(0.292)	(0.327)
year::2019:Ever on Epic	0.380	0.466
	(0.367)	(0.394)
year::2020:Ever on Epic	0.036	0.319
	(0.416)	(0.472)
year::2021:Ever on Epic	0.284	0.745
	(0.435)	(0.503)
year::2022:Ever on Epic	1.317*	2.026**
	(0.561)	(0.677)
year::2023:Ever on Epic	1.215+	2.316**
	(0.621)	(0.783)
year::2024:Ever on Epic	2.175***	3.408***
	(0.580)	(0.756)
Log(Quarters Since Release)	3.071***	2.994***
	(0.151)	(0.207)
Num.Obs.	69654	69654
R2	0.288	0.421
R2 Adj.	0.288	0.413
R2 Within	0.130	0.128
R2 Within Adj.	0.130	0.128

Table 5: Discount Depth: Year-Relative Effects

	No Game FE	Game FE
+ p < 0.1, * p < 0.05, ** p < 0.01, *** p < 0.001		
Reference year 2018.		

Sample Composition Checks

How many months of data for each category

```
reg_data %>%
  group_by(game_title, ever_on_epic) %>%
  summarise(n_months = n(), .groups = "drop") %>%
  group_by(ever_on_epic) %>%
  summarise(mean_months = mean(n_months),
            median_months = median(n_months),
            n_games = n())
```

```
## # A tibble: 2 x 4
##   ever_on_epic mean_months median_months n_games
##         <int>         <dbl>         <dbl>   <int>
## 1             0          82.0           87     544
## 2             1          70.4          70.5     356
```

Treated games are observed for a somewhat shorter panel than controls on average (70 vs. 82 months), but since the treatment interaction is identified off the treated games while the longer panels belong to controls, observation-weighting doesn't tilt the estimated effect.

Treated / Control games by year

```
reg_data %>%
  group_by(year, ever_on_epic) %>%
  summarise(n_games = n_distinct(game_title), .groups = "drop") %>%
  pivot_wider(names_from = ever_on_epic, values_from = n_games,
             names_prefix = "treated_")
```

```
## # A tibble: 11 x 3
##   year treated_0 treated_1
##   <dbl>   <int>   <int>
## 1 2014      92     36
## 2 2015     160     68
## 3 2016     223    104
## 4 2017     280    147
## 5 2018     337    175
## 6 2019     383    197
## 7 2020     420    238
## 8 2021     465    270
## 9 2022     512    310
## 10 2023     544    350
## 11 2024     544    356
```

The sample grows monotonically from 128 games in 2014 to 900 in 2024 with the treated share rising from ~28% to ~39%, but because treated and control arms grow roughly symmetrically and the interaction is a within-year contrast, this cross-year compositional drift affects both arms comparably rather than driving the estimate.